NextJS Tutorial #2 - Rendering User Interfaces

Concepts : How UI Renders in the Browser, React Setup with Vite, JSX & Virtual DOM

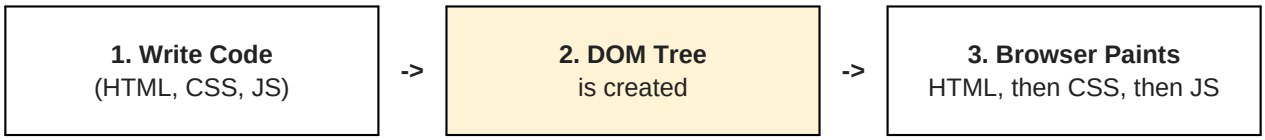
Overview of This Lecture

In the last video we learned the basic difference between React and NextJS. This lecture covers the **foundation of React** - if we skip it, NextJS will feel confusing later.

- How a normal HTML/JavaScript page **renders in the browser** (DOM)
- Setting up a **React project** using Vite
- What is **JSX** and why React needs it
- **Rules of JSX** (className, curly braces, etc.)
- How UI is rendered in the browser in React (**Virtual DOM**)
- Common **interview questions**

How a Normal Page Renders in the Browser

Suppose we have HTML code with a **body** tag, a **div**, an **h1** heading, an unordered list and a button. The browser follows these steps:



- The **DOM (Document Object Model)** is a tree: HTML at the top, then body, then div, then the tags inside the div, and so on
- **DOM is a facility provided by the browser**
- The browser builds the website on screen according to the DOM tree
- This is the normal flow followed in most languages

Setting Up a React Project (Vite)

Here we set up **only React** (not NextJS yet). Steps:

1. Open the official React website and copy the command to create a project with **Vite**
2. Create a folder (e.g. *nextjs-tutorial*) in VS Code, open the terminal and paste the command
3. Enter the **project name** (e.g. *react-tutorial*)
4. Select the framework: **React**
5. Select the variant: **JavaScript** (simple JavaScript)
6. *rolldown-vite* (experimental): choose **No**
7. Install with npm and start now: choose **Yes**

Vite installs the dependencies and starts the project automatically. We get a small **counter app** running (React + Vite), which confirms the setup is done.

```
npm create vite@latest
```

> **Note** - Exact prompts can differ slightly between Vite versions, so always follow the command on the official React/Vite documentation.

What is JSX?

JSX = JavaScript Syntax Extension. It means writing **HTML-like syntax inside JavaScript**.

Why do we need JSX?

In plain JavaScript, creating HTML elements needs **document.createElement**, **innerHTML**, etc. When the UI has a lot of HTML, this becomes messy and confusing. In React we write HTML-like code directly inside JavaScript using JSX.

Practical example (App.jsx)

```
function App() {
  return (
    <h1>React Tutorial</h1>
  );
}

export default App;
```

- Inside **src/App.jsx** we create a function **App** that returns the UI
- We export it with **export default App**; **main.jsx** imports and renders it
- It looks like HTML, but **it is JavaScript**
- React **internally converts JSX into JavaScript**; the browser reads that JavaScript and displays the result

> **Note** - Remember: **JSX = JavaScript Syntax Extension**. It is not HTML - it only looks like HTML.

Rules of JSX

1. Every component has a parent

Every component is used inside some parent component. For example, **App** is used by its parent **main.jsx**: whatever we **export** from one file is **imported** in another.

2. Use *className* instead of *class*

We cannot use the normal **class** keyword in JSX; we write **className**. To use CSS: import the CSS file, then apply the class name.

```
import './App.css';

function App() {
  return <h1 className="header">React Tutorial</h1>;
}
```

```
.header {
  background: red;
}
```

3. JavaScript goes inside curly braces { }

Any JavaScript code/expression written inside JSX must be put **within curly brackets**.

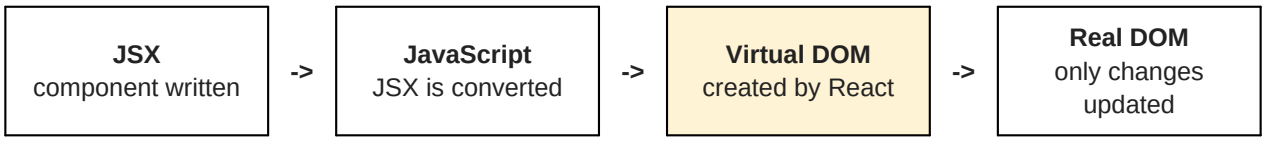
```
function App() {
  let name = "Mohit";
  return <h1>{name}</h1>;   // displays: Mohit
}
```

Rule	Example
Component has a parent	App is imported/rendered by main.jsx
Use className, not class	className="header"
JS inside curly braces	{name}

> **Extra** - Another important JSX rule (not stressed in the video): a component must return **one root element**. To return several elements, wrap them in a single **div** or a Fragment (<> ... </>).

How UI Renders in the Browser (React)

React follows this basic flow:



1. The React component is written in **JSX**
2. React **converts JSX into JavaScript**
3. React creates a **Virtual DOM**
4. The Virtual DOM is **compared with the actual DOM** and the UI is displayed accordingly
5. If the data is the **same as before**, nothing is reloaded; if there is **new data**, only that part is changed
6. The old page is **not reloaded** - only the changed part updates

This is why React is considered fast.

> **Extra** - Technically, React compares the *new* Virtual DOM with the *previous* Virtual DOM (called **diffing**) and then updates only the differences in the real DOM.

Real DOM vs Virtual DOM

Real DOM	Virtual DOM
Created and managed by the browser	A lightweight JavaScript copy of the DOM, created by React
Updating can be slower, since changes may re-render more of the page	Changes are compared first, so only the needed parts are updated
Browser paints the UI from it	Helps React decide what to change in the real DOM

Table based on the lecture's explanation; interviewers usually expect this kind of comparison.

Quick Revision - Interview Questions

Q1. What is JSX?
Ans: JSX is a JavaScript Syntax Extension that lets us write HTML-like code inside JavaScript. React converts it into JavaScript internally.

Q2. What is Virtual DOM?
Ans: A lightweight copy of the real DOM that React creates, used to find what changed so only that part of the UI is updated.

Q3. How will you apply CSS in a React project?
Ans: Create a CSS file, import it in the component, and use className (not class) to apply the styles.

Q4. What is the difference between Real DOM and Virtual DOM?
Ans: Real DOM is the browser's actual tree; Virtual DOM is React's in-memory copy used to update only the changed parts efficiently.

Q5. How is UI rendered in the browser in React?
Ans: JSX is converted to JavaScript, React builds a Virtual DOM, compares it with the previous state, and updates only the changed part of the real DOM.

Q6. How do you use JavaScript inside JSX?
Ans: Write the JavaScript expression inside curly braces { }.

Key Takeaways

- Browser flow: code -> DOM tree -> browser paints the page. DOM is provided by the browser.
- React project can be created quickly with Vite (npm create vite@latest).
- JSX = JavaScript Syntax Extension: HTML-like syntax inside JavaScript.
- JSX rules: parent component, className instead of class, JavaScript inside { }.
- React converts JSX to JavaScript, builds a Virtual DOM and updates only what changed - no full page reload.

> *Keep Learning & Keep Exploring!*